

Dive into Node.js

Full Stack



What is a 'Runtime'?

Chrome's V8 engine and the leap from Browser to Server.



V8 executes JavaScript outside the browser, providing access to system resources (file system, network).

What is Node.js?

An open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside a web browser.



Built on Chrome's V8 JavaScript engine for performance.



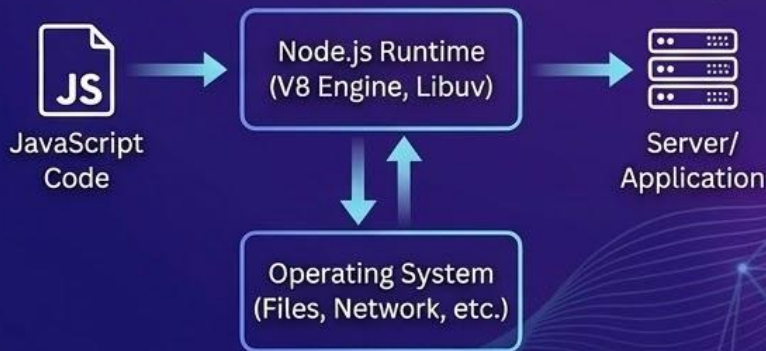
Uses an event-driven, non-blocking I/O model (lightweight & efficient).



Enables server-side scripting and building scalable network applications.



Includes npm, the largest ecosystem of open-source libraries.



Why Node.js Exists?



Unifies web development with a single language (JavaScript) for client and server.



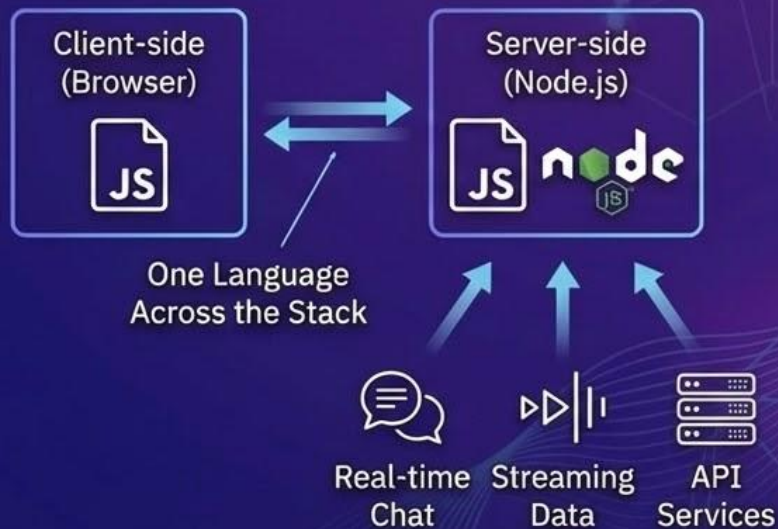
Handles high concurrency and real-time data efficiently via non-blocking I/O.



Provides high performance for I/O-bound applications using Chrome's V8 engine.



Enables rapid development with a vast ecosystem of reusable packages (npm).



Event-Driven Architecture

- Node.js reacts to events.

- Examples of events:

- HTTP request
 - File read completed
 - Database response
 - Timer finished

When an event happens → callback executes

```
server.on("request", handler)
```

Event:
HTTP request

Action:
handler()

Non-Blocking I/O

Blocking code:

read file



(wait)



continue

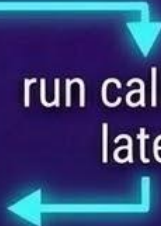
Server freezes.

Non-blocking:

read file



run callback
later



continue execution

Node uses non-blocking I/O so it
can handle thousands of requests.



Callback

A function passed to run later.

```
setTimeout(() => {  
  console.log("Done");  
}, 2000);
```

The arrow function
is the **callback**.

It runs after the timer event.

What is a Thread?



The smallest unit of processing that can be scheduled by an operating system.



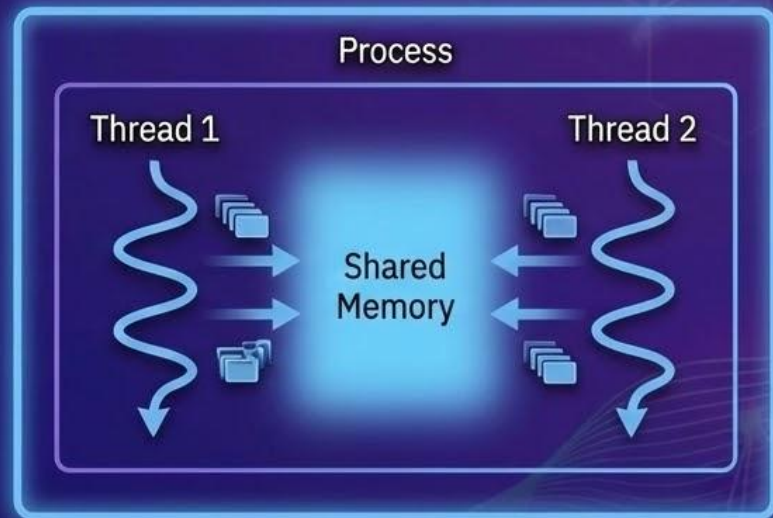
A separate, independent path of execution within a single process.



Threads share the same memory space and resources of their parent process.



Multithreading allows multiple threads to execute concurrently, improving application responsiveness.



The Single-Threaded Model



JavaScript executes code on a single main thread.



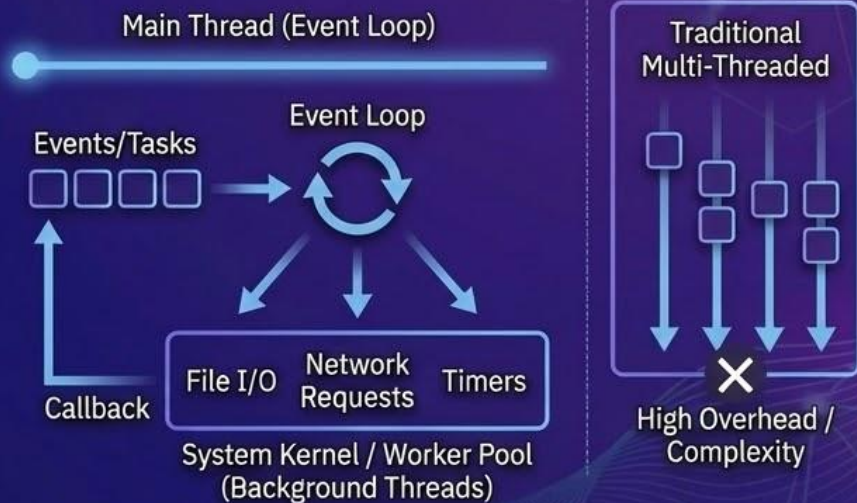
Relies on the Event Loop to manage asynchronous operations non-blockingly.



Avoids the overhead and complexity of traditional multi-threaded context switching.



Long-running, CPU-intensive tasks can block the thread (mitigated by Worker Threads).



Why Full-Stack Uses Multi-Threading?



Handles multiple concurrent user requests efficiently.



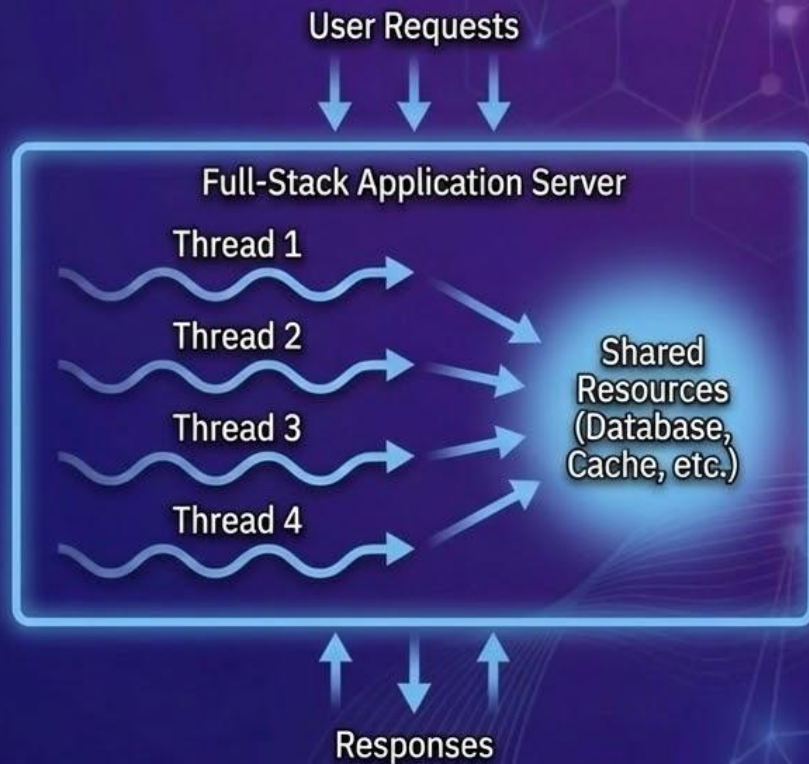
Improves application responsiveness and throughput.



Performs background tasks (e.g., data processing, I/O) without blocking the main thread.



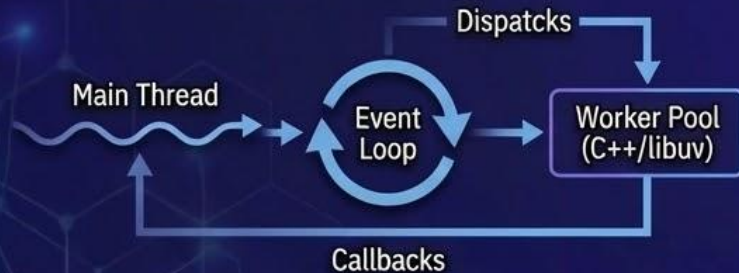
Maximizes utilization of multi-core server hardware.



Node.js Single-Threaded vs. Traditional Multi-Threaded

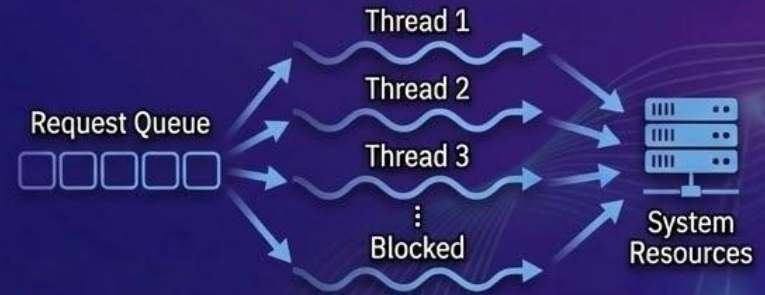
Node.js (Single-Threaded Event Loop)

- Non-blocking I/O operations.
- Efficient resource utilization (low memory overhead).
- No complex thread synchronization or locking needed.
- Highly scalable for concurrent connections.



Traditional Multi-Threaded

- Blocking I/O can stall threads.
- High memory overhead per thread.
- Complex thread management and context switching.
- Limited scalability due to resource constraints.



Node.js's model is often superior for I/O-bound, high-concurrency applications due to its efficiency and low overhead.

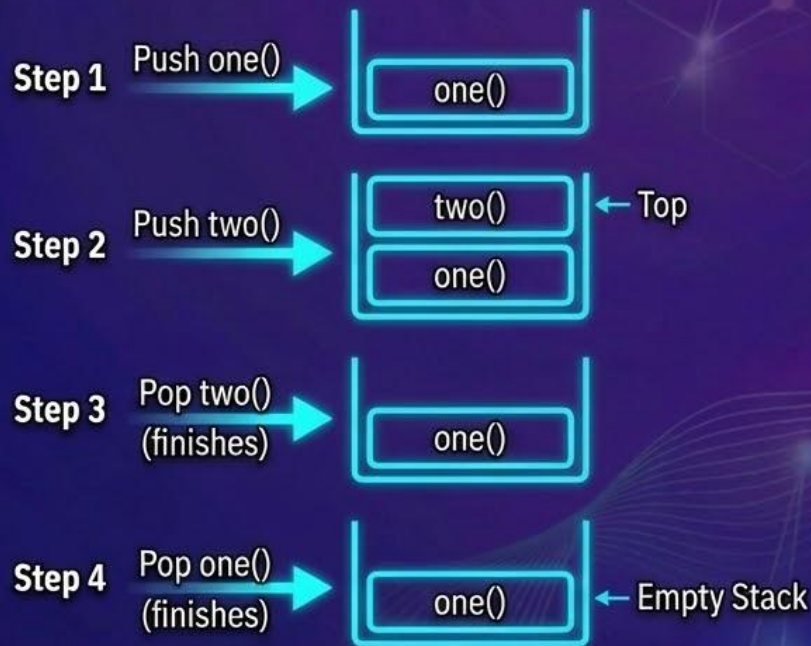
The Call Stack: Execution Manager in JavaScript & Node.js

Stack Rule & Example

- ❖ LIFO (Last-In, First-Out) data structure.
- ❖ The last function pushed is the first one popped.

```
function one() {  
  two();  
}  
  
function two() {  
  console.log("Hello");  
}  
  
one();
```

Execution Stack Visualization



The Call Stack manages function execution context.

The Event Queue: Managing Asynchronous Operations in Node.js

Queue Rule & Example

- FIFO (First-In, First-Out) data structure.
- First item added → first item removed.

Example (Adding):

Add A

Add B

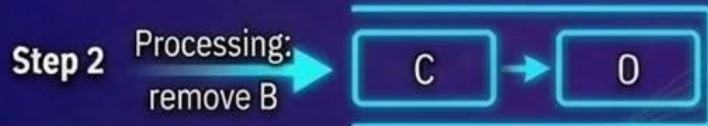
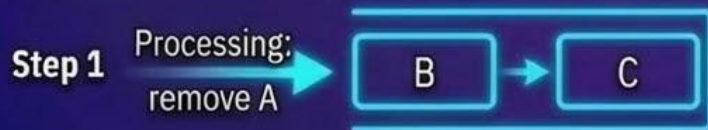
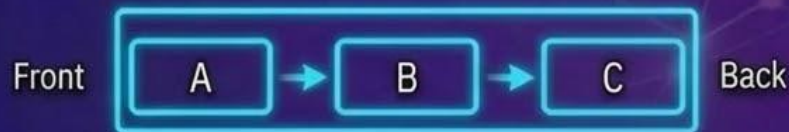
Add C

Queue looks like:

A → B → C

(Front → Back)

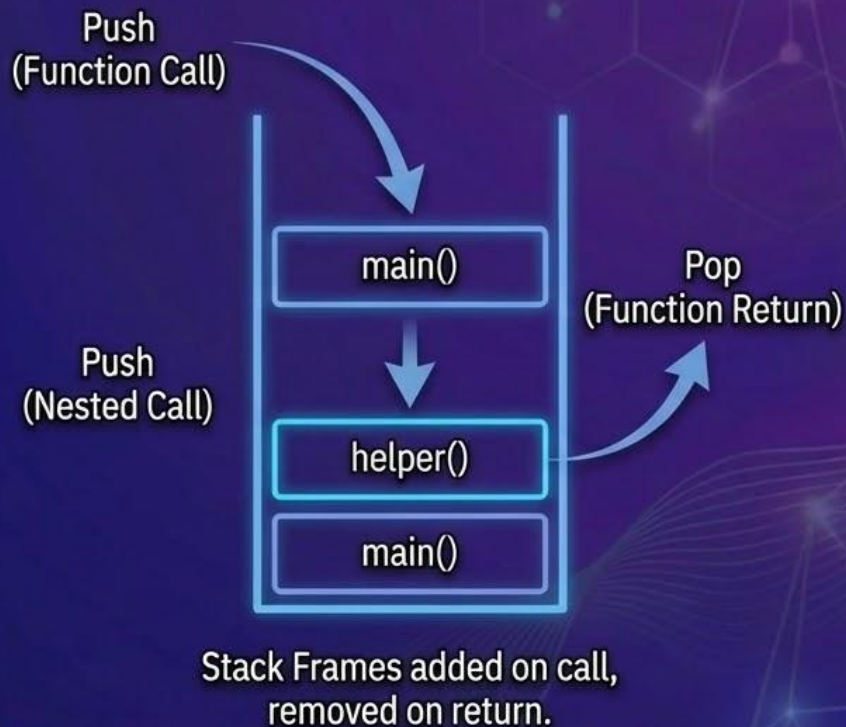
Queue Processing Visualization



Exactly like a line at a ticket counter.

The Call Stack: Managing Execution Context

- ❖ LIFO (Last-In, First-Out) data structure.
- ❖ Records where in the code the execution is.
- ❖ Stores stack frames (variables, return address) for active function calls.
- ❖ Essential for single-threaded execution flow.
- ❖ A blocking operation stalls the entire stack.

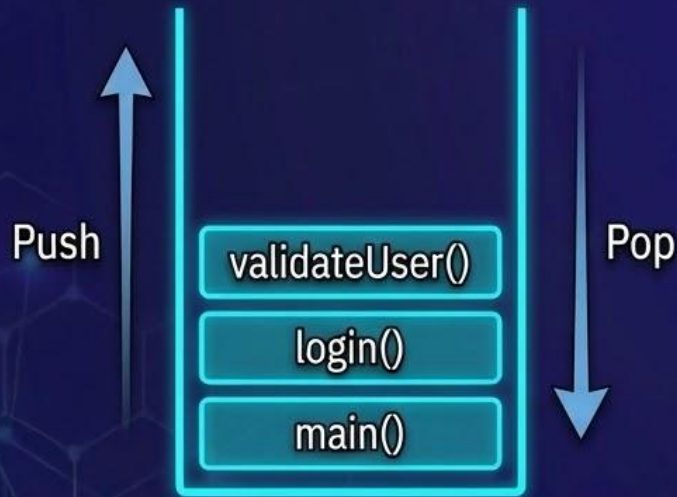


Node.js: Call Stack vs. Event Queue & The Event Loop

In Node.js both exist but serve different roles.

Call Stack

- Handles function execution



Event Queue

- Stores callbacks waiting to execute

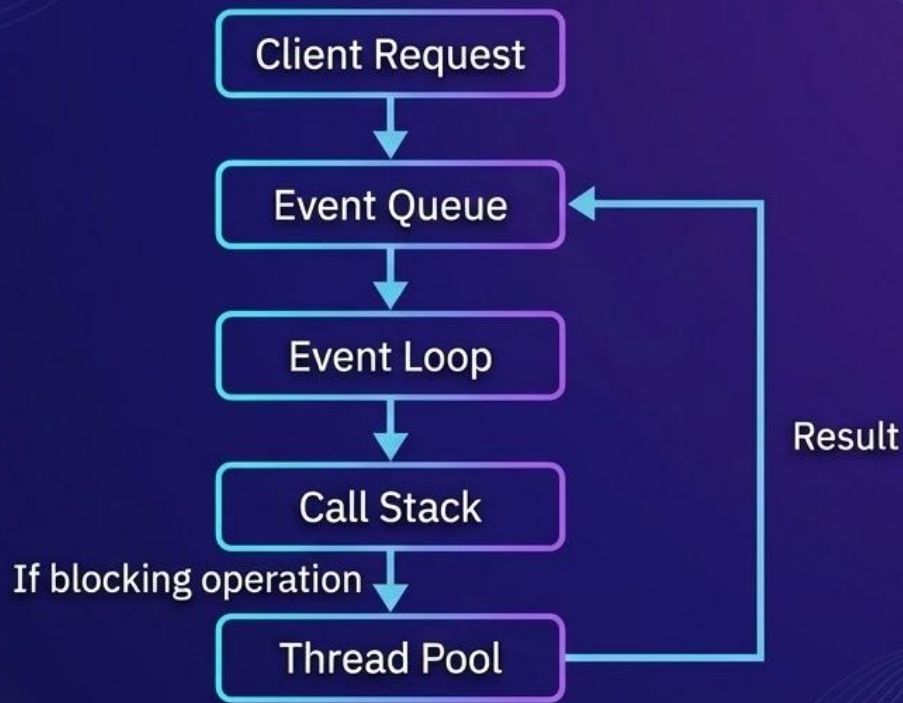
```
setTimeout(() => {  
  console.log("Done");  
}, 2000);
```

Flow:

1. JS executes code
2. Timer runs in background
3. Callback enters Event Queue
4. Event Loop checks stack
5. If stack empty → move callback to stack



The Real Workflow (Simple Architecture)



This is the actual runtime flow.

Step-by-Step Workflow

Example code:

```
const fs = require("fs");  
console.log("Start");  
fs.readFile("data.txt", () => {  
  console.log("File Read Completed");  
});  
console.log("End");
```

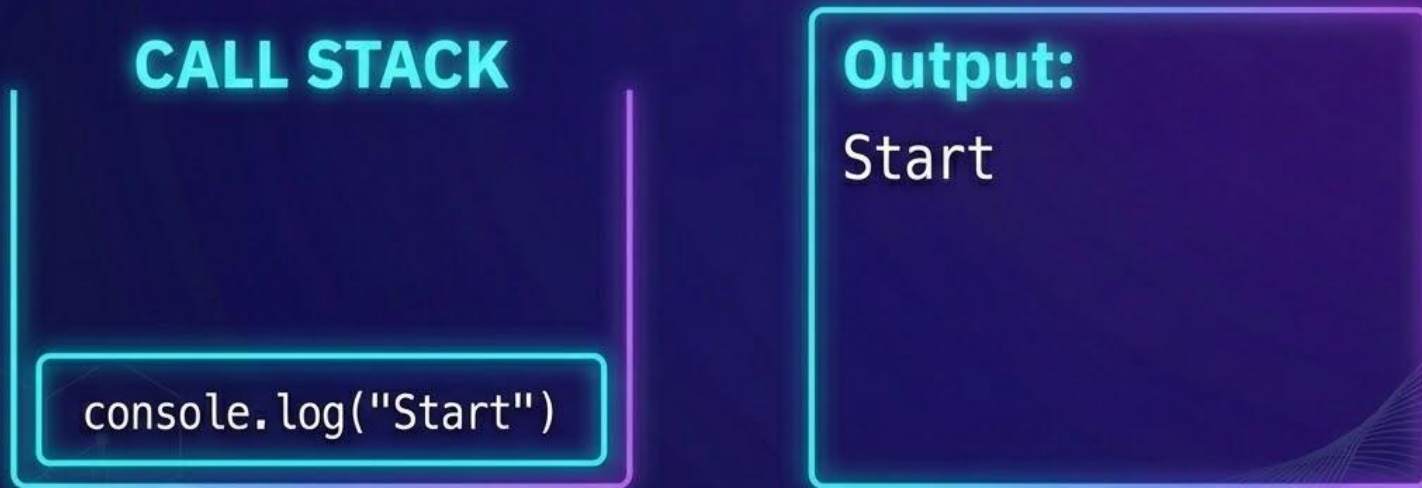
Execution order:

- 1. Start
- 2. End
- 3. File Read Completed

Why? Let's break it down.

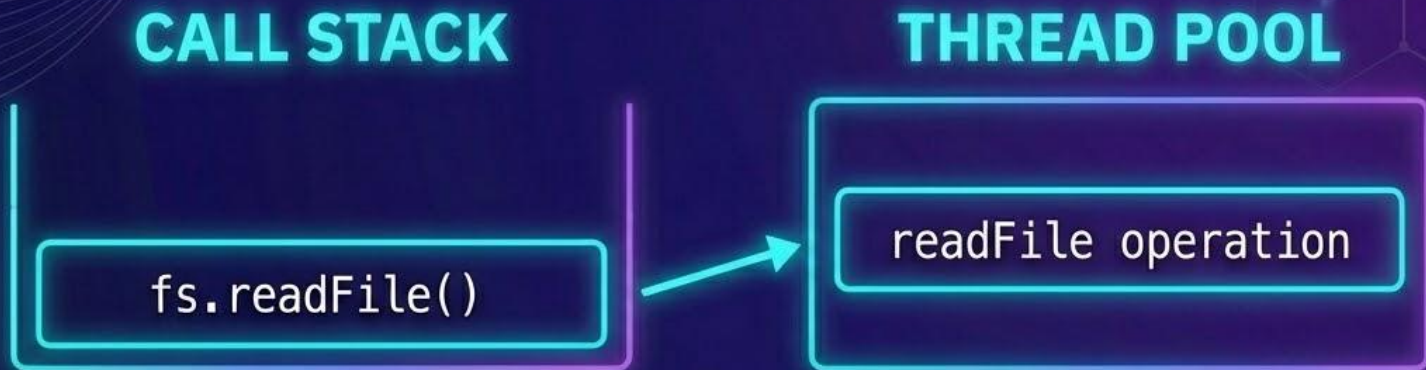
Step 1 — Code Enters Call Stack

■ The Call Stack executes JavaScript.



Then removed from stack.

Step 2 – Async Operation Appears



- fs.readFile() is I/O work.
- Node does NOT execute it in the call stack.
- Instead it sends it to the Thread Pool managed by libuv.

Stack becomes free.

Step 3 — Next Code Runs

Now stack continues.

CALL STACK

```
console.log("End")
```

Output:

End

Stack empty.

Why Node.js Is Called Single-Threaded & The Hidden Truth

The Single-Threaded Model

- JavaScript execution happens in one main thread.
- Only one function runs at a time in the Call Stack.

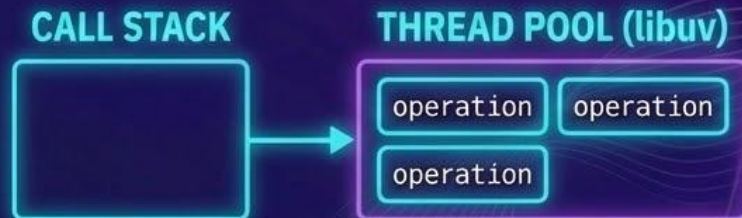


That's the single-threaded model of Node.js.

But Here's the Part Beginners Miss...

Node.js is **not truly** single-threaded internally. It uses multiple threads behind the scenes through libuv.

- Example operations handled by thread pool:
 - File system
 - DNS lookup
 - Compression
 - Some crypto operations



So architecture looks like this:

Step 4 – File Read Finishes



- Thread pool finishes work.
- Callback moves to Event Queue.